Insertion Sort

AlgoExpert · Easy · Sorting

The problem

- Grow a sorted prefix by sinking each new element into place.

Intuition

- Grow a sorted prefix, inserting each new element into its correct position by shifting it left.

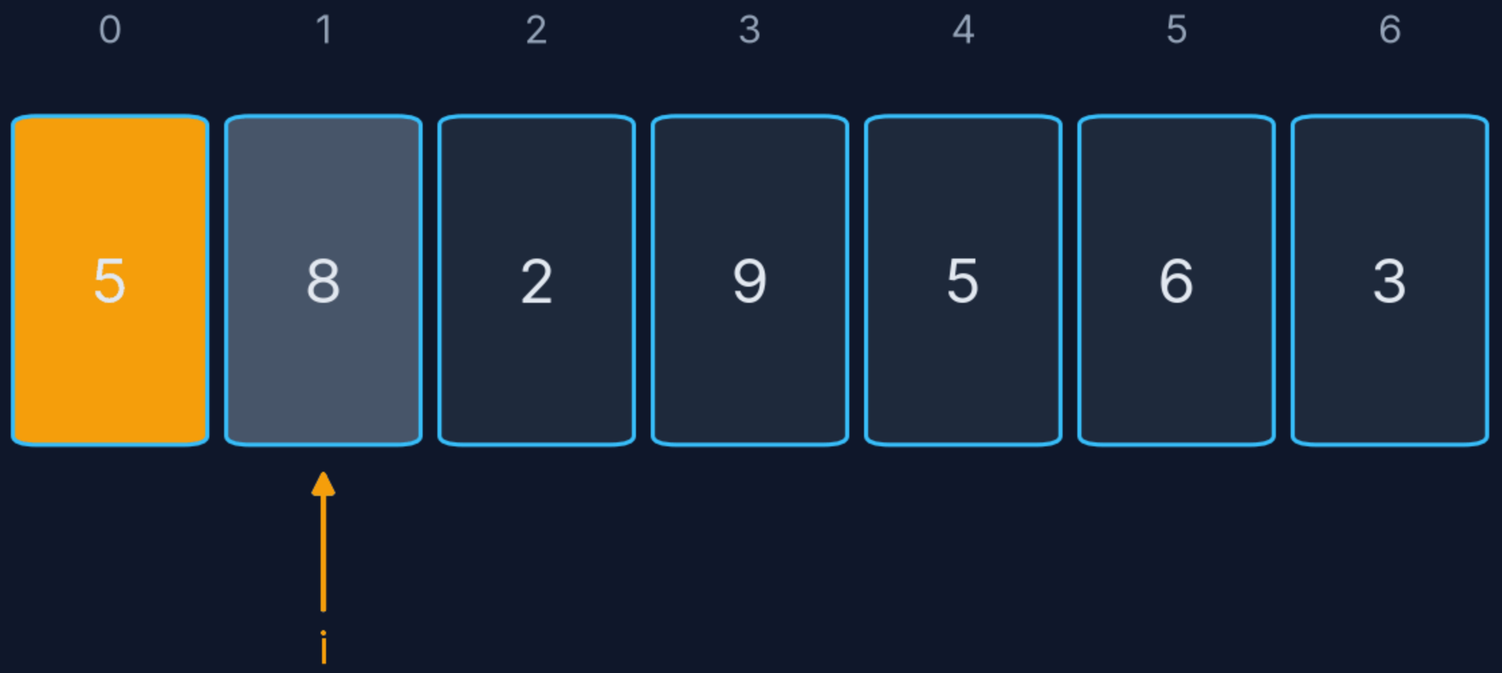
Brute force vs optimal

- $O(n^2)$ in the worst case (reverse sorted)
- O(n) on nearly-sorted data — its big advantage
- Used inside hybrid sorts like TimSort for small runs

Approach

- 1. For each element from the second
- 2. Swap it left while it's smaller than its neighbour
- 3. The prefix stays sorted

Worked example



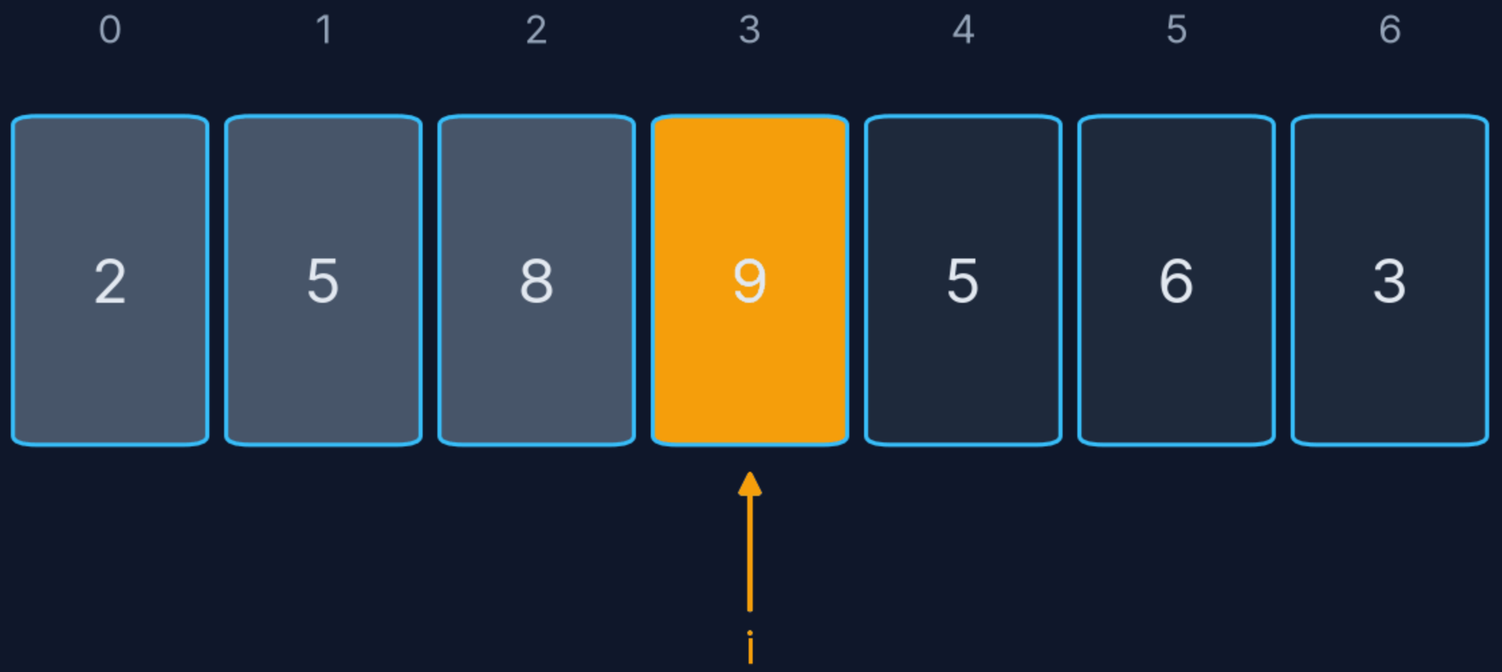
Insert into sorted prefix → lands at index 0

Worked example



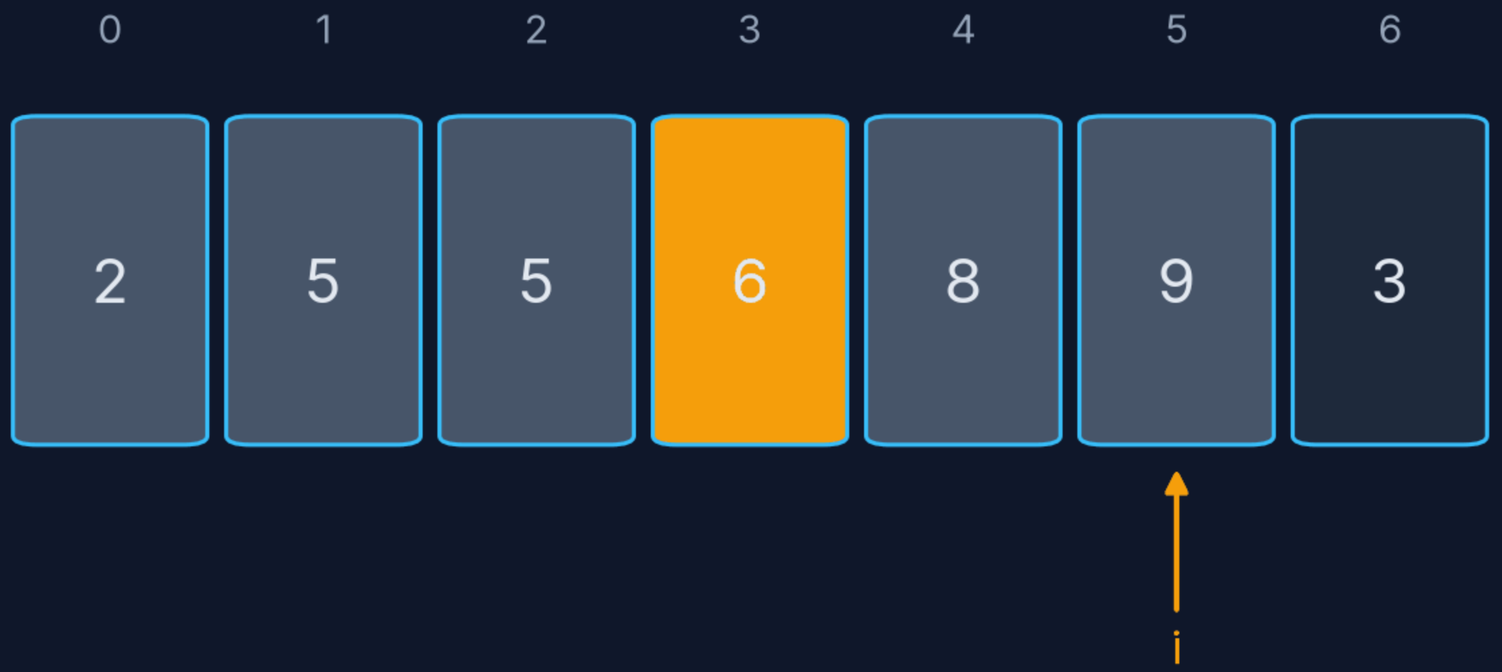
Insert into sorted prefix → lands at index 0

Worked example



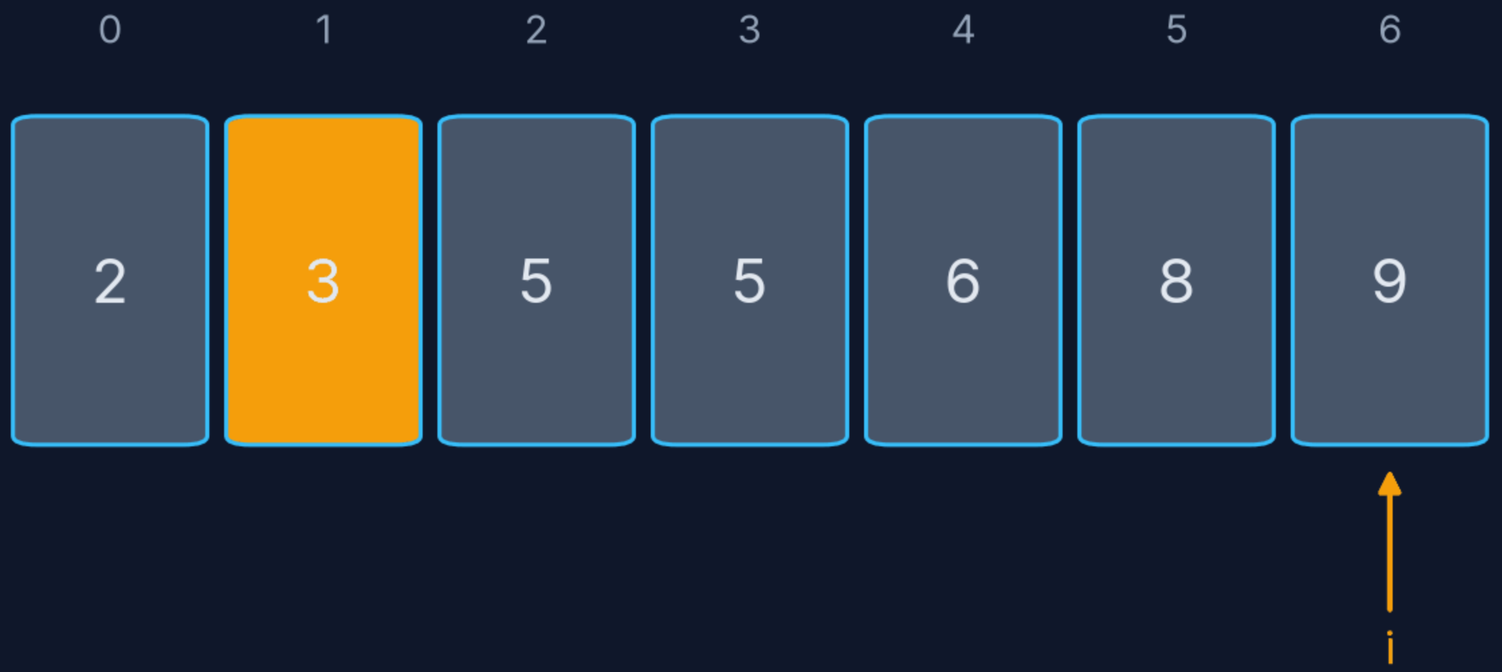
Insert into sorted prefix → lands at index 3

Worked example



Insert into sorted prefix → lands at index 3

Worked example



Insert into sorted prefix → lands at index 1

Java solution

```java
public class InsertionSort {
    public static int[] insertionSort(int[] a) {
        for (int i = 1; i < a.length; i++) {
            int x = a[i], j = i - 1;
            while (j >= 0 && a[j] > x) { a[j + 1] = a[j]; j--; }
            a[j + 1] = x;
        }
        return a;
    }
}
```

Complexity

- Time: O(n^2) worst, O(n) best
- Space: O(1)

Edge cases & pitfalls

- Compare with strict < to stay stable
- Start from index 1
- Shift instead of swap to save writes

Interview tips & follow-ups

- Explain the sorted-prefix invariant
- Follow-up: binary search for the insert position

Recap

- Grow a sorted prefix, inserting each new element into its correct position by shifting it left.
- Complexity: O(n^2) worst, O(n) best